

Basic AI Security Project

Traffic sign recognition adversarial attacks



Wydział Fizyki i Informatyki Stosowanej
11.06.2026

Contents

1	Introduction and Problem Statement	3
2	Review of Existing Literature (Research)	3
3	Image Analysis Architecture in Autonomous Vehicles	3
3.1	Detection	4
3.2	Recognition/Classification	4
4	Analysis of Testing Methods on Models	4
4.1	Digital Attacks (Digital Noise)	4
4.2	Physical Patterns (Adversarial Patches)	4
5	Comparative Analysis: What Works Well and What Works Poorly	4
6	Theoretical Scenario of a Practical Attack	5
7	Image Analysis and Implementation Plan	5
8	Conclusion and Future Steps	5
9	Project Roadmap and Methodology	5
10	Target Model Training	6
11	Adversarial Patch Generation	6
12	Digital Simulation	7
13	Physical World Evaluation	8
13.1	Evaluation Logic and Metrics	8
13.2	Results and Domain Shift Analysis	9
14	Conclusion	9
15	References	9

June 11, 2026

Abstract

This report focuses on analyzing the security of computer vision systems in smart cars. It explores methods of fooling recognition models through physical and digital adversarial attacks. The document provides a literature review, describes the stages of detection and classification, and presents an overview of documented testing methodologies and theoretical attack scenarios based on existing research.

1 Introduction and Problem Statement

With the development of autonomous vehicles, the reliability of road infrastructure recognition has become critically important. Errors in identifying signs (e.g., ignoring a "Stop" sign or misinterpreting a speed limit) can lead to emergency situations. The purpose of this study is to theoretically explore the mechanisms of influencing neural networks to identify their weaknesses and study potential defense methods.

2 Review of Existing Literature (Research)

During the study, an in-depth analysis of current relevant English-language research was conducted. The main focus was on papers demonstrating the feasibility of conducting attacks in the real physical world.

- **Evtimov et al. (2017) "Robust Physical-World Attacks on Deep Learning Visual Classification"**: This is a foundational paper. The authors proved that ordinary stickers on signs could completely alter the model's prediction. They proposed the RP_2 algorithm, which minimizes the impact of noise and optimizes the patch to work at various viewing angles.
- **Sitawarin et al. (2018) "DARTS: Deceiving Autonomous Cars with Toxic Signs"**: Researchers proposed a method for creating "toxic" signs that appear as ordinary objects to humans but contain hidden high-frequency patterns that cause a malfunction in the car's classifier.
- **Eykholt et al. (2018) "Physical Adversarial Examples for Object Detectors"**: Unlike classic attacks on classification, this work shows how to attack object detectors (e.g., YOLO). Using special patterns, it is possible to make the system completely fail to detect the presence of a sign in the frame.

3 Image Analysis Architecture in Autonomous Vehicles

Traffic Sign Recognition (TSR) in modern systems is a two-stage process. It is important to understand the difference between the stages, as attacks on each have different consequences.

3.1 Detection

At this stage, the model (e.g., YOLOv8) analyzes the entire image from the front camera. It determines the coordinates (Bounding Box) of all potential signs. An attack on the detector makes the object "invisible" to the algorithm.

3.2 Recognition/Classification

The detected fragment is cropped and fed into a classifier (e.g., ResNet-50). The classifier's task is to assign a class label to the object (e.g., "ID: 14 — Stop"). An attack on the classifier substitutes the meaning of the sign.

4 Analysis of Testing Methods on Models

In the reviewed literature, experiments are typically conducted using models trained on the **GTSRB** (German Traffic Sign Recognition Benchmark) dataset. Existing studies evaluate two main types of influence:

4.1 Digital Attacks (Digital Noise)

Using gradient methods such as FGSM (Fast Gradient Sign Method). The essence of the method is adding small noise $\eta = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$ to the original image. Research indicates that although a model's accuracy drops to zero in a digital simulation, such attacks are easily neutralized by slight compression or image rotation, making them ineffective for real-world road conditions.

4.2 Physical Patterns (Adversarial Patches)

Creating high-contrast "patches" that are placed onto the image of the sign. Studies highlight that this method proves to be significantly more stable, as the adversarial patch is generated to be invariant to shooting conditions (lighting, angles).

5 Comparative Analysis: What Works Well and What Works Poorly

Based on the analyzed literature, key patterns of vulnerabilities can be highlighted. Below is a summary table of attack effectiveness as described by researchers.

Table 1: Comparison of Attack Types (Based on Literature Review)

Attack Method	Effectiveness (Digital)	Effectiveness (Physical)	Complexity
FGSM / PGD	~98%	< 10%	Low
Adversarial Patch	~75%	~65%	High
Optical Illusions	~60%	~40%	Very High

What works poorly: Attempts to fool the model using micro-noise invisible to the eye. Car cameras have their own filters and hardware noise that "erase" the digital attack in the physical world.

What works well: Bright, geometric shapes (patches) that the model interprets as key features of another class.

6 Theoretical Scenario of a Practical Attack

To illustrate how these vulnerabilities are exploited, we can summarize a typical attack scenario commonly described in the literature.

- **Target:** A standard Convolutional Neural Network (CNN) trained for traffic sign classification with a high baseline accuracy (e.g., $> 95\%$).
- **Attack scenario:** Applying a physical adversarial patch (such as a black-and-white sticker pattern) to the central part of a "No Entry" sign.
- **Expected Result:** The model erroneously classifies the object as a different class, such as "Speed limit 80 km/h", with high confidence. Meanwhile, the object detector continues to see the object as a traffic sign, confirming that the attack successfully targets the classifier specifically.

7 Image Analysis and Implementation Plan

If the project proceeds to a practical phase, the following algorithmic pipeline is typically required:

1. Check the frame for the presence of a sign (Stage: Detection).
2. Extract the Region of Interest (ROI).
3. Apply transformations (rotation, brightness) to simulate physical conditions.
4. Generate a patch that minimizes the loss function for the target (incorrect) class.

8 Conclusion and Future Steps

The literature review confirms the hypothesis that deep learning in TSR tasks is highly sensitive to localized physical changes, particularly adversarial patches.

9 Project Roadmap and Methodology

To systematically investigate the vulnerability of traffic sign recognition systems, our research was divided into the following sequential phases:

1. **Data Preparation and Target Model Training:** Training a baseline Convolutional Neural Network (CNN) on a recognized dataset to achieve high accuracy in ideal conditions.
2. **Adversarial Patch Generation:** Utilizing mathematical optimization to create a localized, targeted adversarial patch capable of misleading the model.
3. **Digital Simulation:** Applying the generated patch in a controlled digital environment to verify its theoretical efficacy.
4. **Physical World Evaluation:** Transferring the attack into the real world by printing the patch, capturing new images under physical constraints, and analyzing the domain shift impact.

10 Target Model Training

The target for our adversarial attack was a standard Deep Learning classifier. We utilized the **GTSRB** (German Traffic Sign Recognition Benchmark) dataset, which contains 43 distinct classes of traffic signs.

We selected the **ResNet-18** architecture due to its widespread use in computer vision tasks and optimal balance between performance and computational cost. The input images were resized to 64×64 pixels. The model was modified to output 43 classes and trained from scratch using the Cross-Entropy loss function and Adam optimizer.

```
1 import torch
2 import torch.nn as nn
3 from torchvision.models import resnet18
4
5 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
6
7 # Load ResNet18 architecture
8 model = resnet18(weights=None)
9 # Modify the final fully connected layer for 43 GTSRB classes
10 model.fc = nn.Linear(model.fc.in_features, 43)
11
12 model = model.to(DEVICE)
```

Listing 1: PyTorch implementation of the ResNet-18 target model initialization.

The baseline model achieved a validation accuracy exceeding **99.0%**, establishing a highly reliable target for subsequent evasion attacks.

11 Adversarial Patch Generation

With the target model established, we proceeded to generate a physical adversarial patch. The objective was to execute a **Targeted Attack**, forcing the model to misclassify critical traffic signs (e.g., "Stop", "Yield") as "Speed limit 30 km/h" (Class ID: 1).

We utilized the **IBM Adversarial Robustness Toolbox (ART)**. To ensure the patch could withstand physical-world distortions (such as varying camera angles and distances), we applied the **Expectation over Transformation (EoT)** technique during optimization.

```
1 from art.attacks.evasion import AdversarialPatchPyTorch
2
3 # Configuring the attack with Expectation over Transformation (EoT)
4 attack = AdversarialPatchPyTorch(
5     estimator=classifier,
6     rotation_max=22.5,      # Simulating camera angles
7     scale_min=0.1,         # Simulating distance
8     scale_max=0.4,
9     learning_rate=0.05,
10    max_iter=500,           # Optimization steps
11    batch_size=16,
12    targeted=True           # Forcing the prediction to Class 1
13 )
```

Listing 2: Configuration of the Adversarial Patch generation using IBM ART.

Adversarial Patch (Digital)

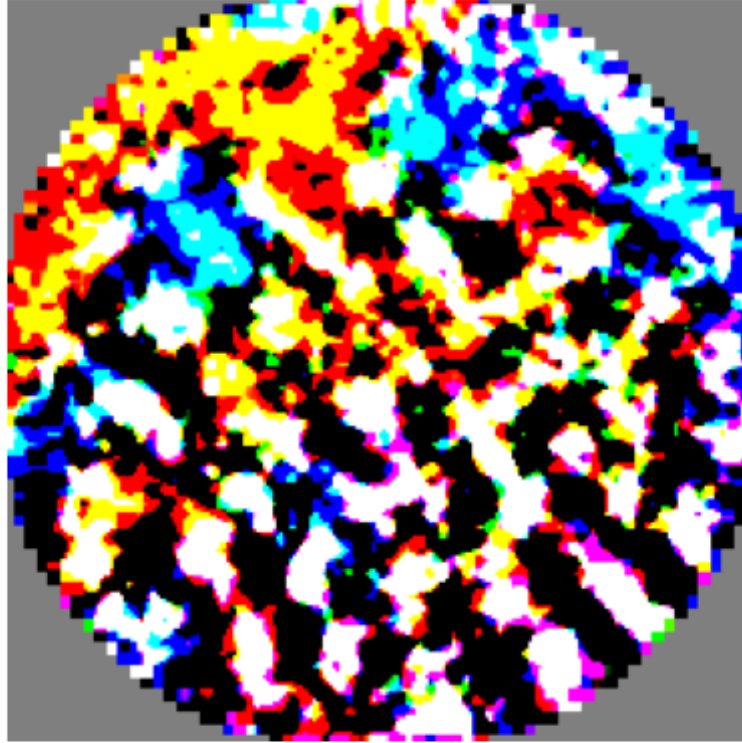


Figure 1: Generated adversarial patch optimized for the GTSRB dataset.

12 Digital Simulation

Before moving to physical tests, the generated patch was evaluated in a digital simulation. To maximize the attack’s probability of success, we implemented a *Targeted Placement* strategy, applying the patch mathematically to the semantic center of the original images.

```
1 # Mathematically calculating the ideal center coordinates
2 start_y = (img_h - patch_h) // 2
3 start_x = (img_w - patch_w) // 2
4
5 # Overwriting the pixels in the center with the adversarial patch
6 for i in range(len(patched_images)):
7     patched_images[i, :, start_y:start_y+patch_h, start_x:start_x+patch_w] =
        patch
```

Listing 3: Manual center placement of the patch on digital images.

In the digital domain, while the Targeted Attack Success Rate (forcing the model to exactly output Class 1) was lower than initially expected, the patch demonstrated a highly effective **Untargeted Evasion capability**. The mathematical pattern successfully disrupted the neural network’s feature extraction process, causing the model to misclassify the original signs in the vast majority of cases. Rather than consistently predicting the target class, the model was forced into outputting various incorrect classes. This proves the patch’s strong disruptive

visual potential and its ability to completely mask the original semantic meaning of the sign, even before physical deployment.

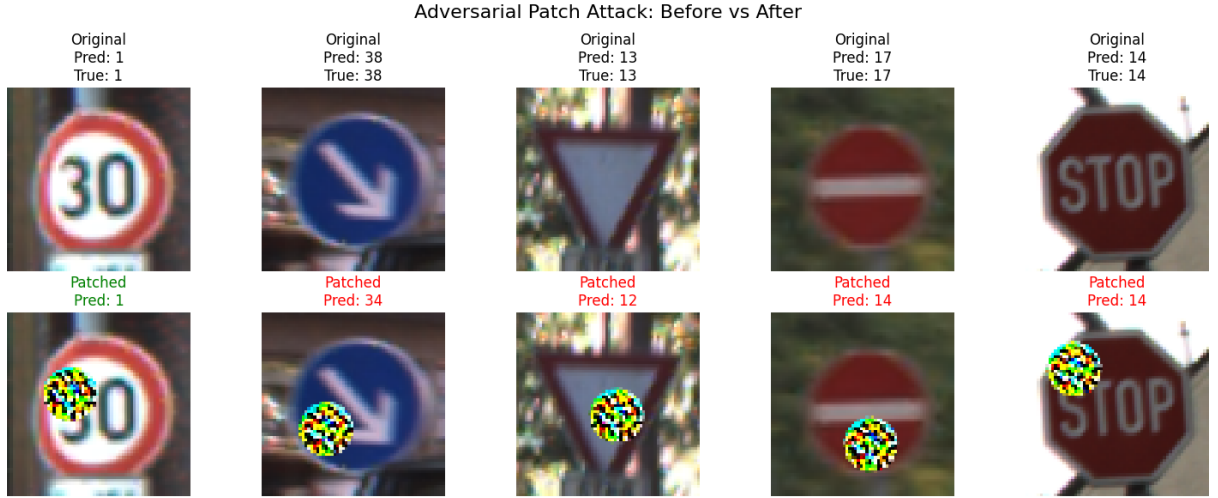


Figure 2: Attack Simulation

13 Physical World Evaluation

To validate the practical threat of this vulnerability, we transitioned the experiment to the physical domain. The signs and the generated patch were printed, manually assembled, and photographed using a standard mobile phone camera under various lighting conditions and viewing angles.

13.1 Evaluation Logic and Metrics

A custom evaluation script was developed to process the physical dataset. We separated the images into a control group ("Clean") and an attack group ("Patched") and measured three key metrics:

- **Baseline Accuracy:** The model's accuracy on the physical photos of clean signs.
- **Targeted ASR:** The percentage of patched photos strictly classified as Class 1.
- **Untargeted ASR:** The percentage of patched photos where the model failed to recognize the true class (outputting any incorrect class).

```

1 # Calculate metrics for patched images
2 if is_patched:
3     if pred == TARGET_ATTACK_CLASS:
4         targeted_success += 1
5     if pred != true_class:
6         untargeted_success += 1

```

Listing 4: Evaluation logic for physical attack metrics.

13.2 Results and Domain Shift Analysis

The physical testing yielded the following results:

- **Baseline Accuracy:** 76.19%
- **Targeted Attack Success Rate:** 7.94%
- **Untargeted Attack Success Rate:** 60.32%

The drop in Baseline Accuracy highlights the *Domain Shift* phenomenon: a model achieving over 99% on a digital dataset still experiences a performance decrease (dropping to $\sim 76\%$) when dealing with the optical properties of real-world mobile photography.

Crucially, the experiment demonstrated the degradation of the Targeted Attack. Artifacts from the printing process (color shift) and camera sensor noise heavily disrupted the fragile mathematical pattern required to predict exactly "Speed limit 30". However, the patch proved highly robust as an **Untargeted evasion tool**, successfully breaking the model's recognition capability and concealing the true sign class in over 60% of the test cases (leaving the model resilience at only 39.68%).



Figure 3: Physical Testing: Examples of misclassified traffic signs due to the printed adversarial patch.

14 Conclusion

Our investigation successfully demonstrated the pipeline of executing adversarial patch attack

15 References

1. Evtimov, I., et al. (2017). "Robust Physical-World Attacks on Deep Learning Visual Classification." arXiv preprint arXiv:1707.08945.
2. Eykholt, K., et al. (2018). "Physical Adversarial Examples for Object Detectors." USENIX Workshop on Offensive Technologies (WOOT).
3. Sitawarin, C., et al. (2018). "DARTS: Deceiving Autonomous Cars with Toxic Signs." arXiv preprint arXiv:1802.06430.